

Procesador RV32I en Simics

Mario Monge

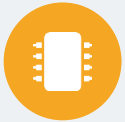
Kevin Wang

Agenda

Temas de hoy

1. Resumen del curso
2. Documentación de Simics
3. Cómo simula Simics un procesador
4. Compilar, ejecutar y cargar programas
5. Archivos de trabajo
6. Código
7. Comandos
8. Cómo probar sus procesadores
9. Problemas comunes

Qué van a construir



Core de un ciclo

`rv32-atomic`

Cada instrucción se busca, se ejecuta y se completa en un mismo ciclo.



Core pipelined

`rv32-pipeline`

Procesador con las cinco etapas definidas

RV32I: 30 instrucciones (aritméticas, lógicas, branches y saltos). Todo programa termina con **`ebreak`**.

Qué van a recibir

```
academia-collaboration/
├── README.md
├── CONVENTIONS.md
├── Instalacion-Simics-Windows.pdf
├── Presentacion-rv32i.pdf
├── modules/
│   ├── rv32-atomic/
│   └── rv32-pipeline/
└── targets/
    └── rv32i/
```

https://github.com/mmongeo/academia-collaboration/tree/rv32i-project

Documentación de Simics

En el proyecto

`documentation.bat`

Una vez ejecutado:

La carpeta **`win64\doc-index`**
Contiene la documentación. Pueden abrir **`index.html`** para visualizarla en el navegador. O solamente vuelven a ejecutar **`documentation.bat`**

Ayuda desde la línea de comandos

```
simics> help pregs
simics> help-search pregs
simics> apropos breakpoint
simics> api-help SIM_LOG_INFO
```

`api-help` explica las funciones de la API en C, como `SIM_LOG_INFO` o los asignadores `MM_*`.

Cycles y steps

Cycle

Tiempo de reloj. Utilizar helper **`increment_cycle()`**

Step

Una instrucción completada. Utilizar helper **`increment_step()`**

Comando de CLI **`ptime`** muestra los dos contadores.

El cosimulador

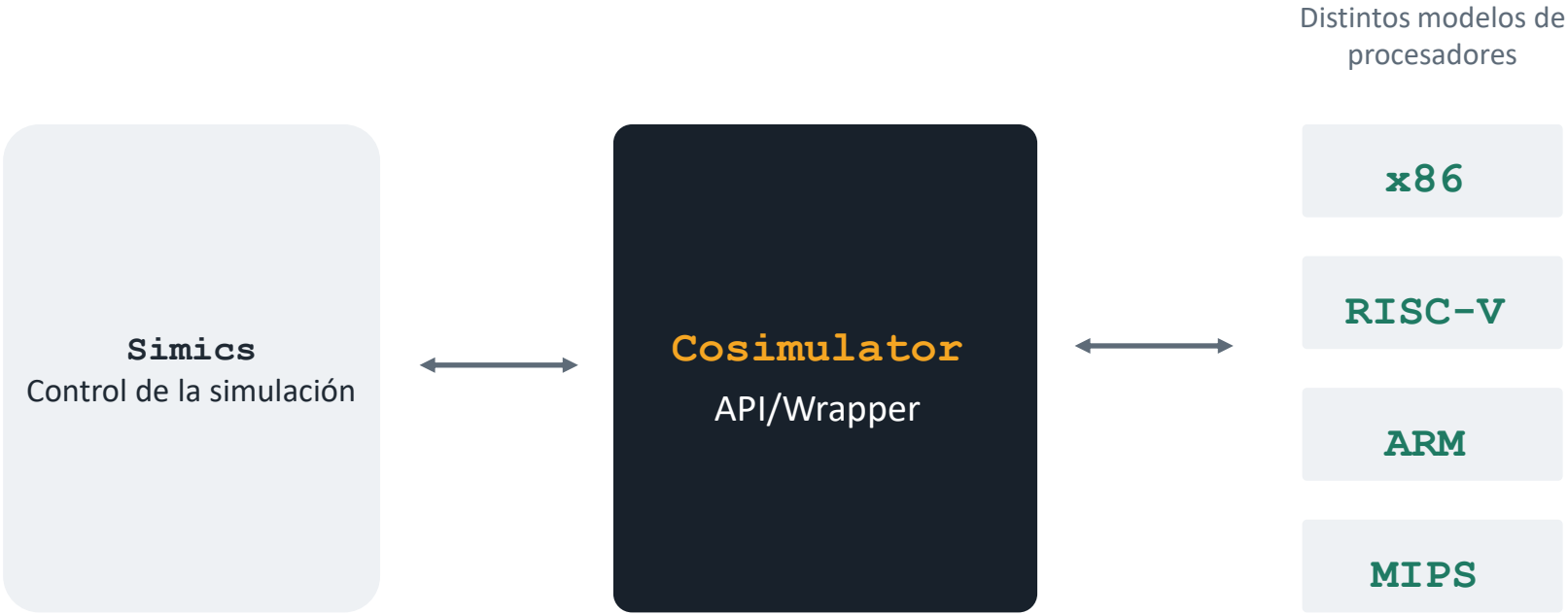
Simics
Control de la simulación

Tengo varios modelos ya hechos:
¿Cómo los uso con Simics?

Distintos modelos de procesadores

- **x86**
- **RISC-V**
- **ARM**
- **MIPS**

El cosimulador



Formato de los programas

```
addi x1, x0, 3     # 0x00
addi x1, x1, -1    # 0x04
bne  x1, x0, -4    # 0x08  salta a 0x04
jal  x5, 12        # 0x0c  salta a 0x18
addi x2, x0, 1     # 0x10
ebreak             # 0x14
addi x3, x0, 7     # 0x18
jalr x0, 0(x5)     # 0x1c  salta a x5 + 0
```

- Archivo .txt
- Una instrucción por línea
- # empieza un comentario
- branches y saltos: desplazamiento en bytes desde la instrucción
- Se carga desde la dirección 0x0
- Todos los detalles: CONVENTIONS.md

Compilar los módulos

```
bin\make.bat rv32-atomic

bin\make.bat rv32-pipeline
```



Recompilar y reiniciar Simics

Después de cambiar un .c, un .h o el `txt_loader.py`. Simics usa el módulo compilado, no sus archivos.

Ejecutar un programa

```
simics.bat targets\rv32i\rv32-atomic.simics
program=targets/rv32i/programs/all-instructions.txt
```

Para el core pipelined: `rv32-pipeline.simics`

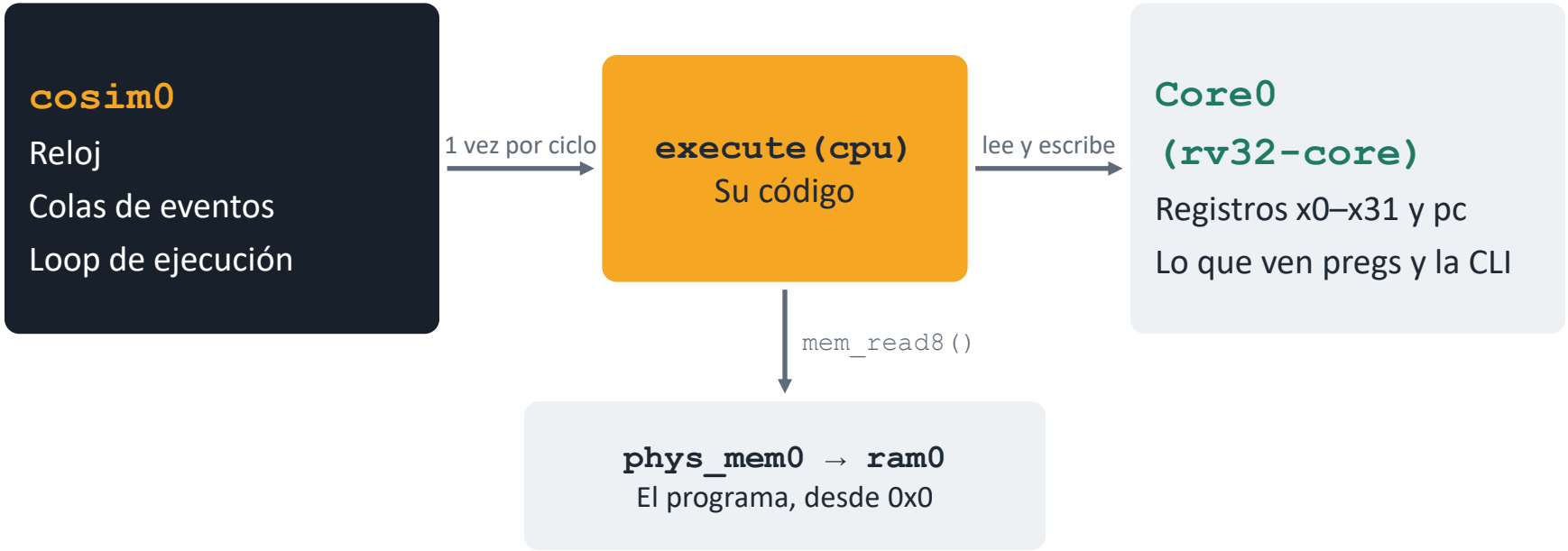
```
simics> run
simics> pregs
simics> quit
```

program=	El programa .txt. Si no se define, se carga all-instructions.txt por defecto
run	Ejecuta hasta el ebreak
pregs	Muestra los registros y el pc
quit	Salir de Simics

Los archivos

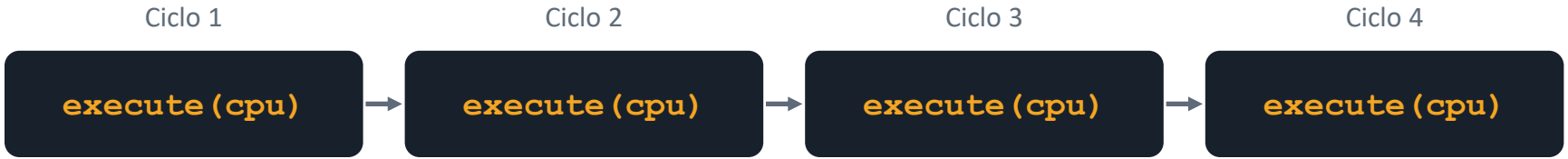
Archivo	Qué contiene	¿Modificar?
`rv32-atomic.c` `rv32-pipeline.c`	Su núcleo: execute() y rv32_disassemble()	Sí
`txt_loader.py`	Su cargador: load_program()	Sí
`rv32-core.h`	rv32_t: el estado del núcleo (pc, x0–x31)	Para añadir su estado
`rv32-core.c`	rv32_new_instance(), rv32_delete_instance() y la conexión con Simics	Solo new y delete
`rv32-helpers.h`	Las funciones que pueden usar	Si desean
`cosim*, event-queue*`	La infraestructura de Simics	No

El cosimulador en ejecución



*Por eso **ptime** muestra cosim0 como procesador: el reloj está en cosim0.*

execute(): se llama una vez por ciclo



`increment_cycle()`

Si no se llama, el tiempo no avanza y Simics se encicla.

`increment_step()`

Una vez por instrucción completada. Máximo una vez por execute(). Si no, da un error de ASSERT

Como manejar ebreak e instrucción ilegal



ebreak

Cuenta el step. El pc no cambia.



Instrucción ilegal

No cuenta step. El pc no cambia.

Funciones de rv32-helpers.h

`reg_read_x(cpu, n)`

Lee xN. x0 siempre vale 0.

`reg_write_x(cpu, n, value)`

Escribe xN. Las escrituras a x0 se ignoran.

`mem_read8(cpu, addr)`

Lee un byte de memoria.

`ebreak(cpu)`

Detiene la simulación cuando execute() retorna.

`illegal_instruction(cpu, pc, instr)`

Registra el error y detiene la simulación cuando execute() retorna.

`increment_cycle(cpu)`
`increment_step(cpu)`

Avanzan el contador de ciclos y el de steps.

Variables y desensamblado

`rv32_new_instance()`
`rv32_delete_instance()`

- Añadan sus campos a rv32_t
- Inicialícenlos en new_instance
- Liberen lo reservado en delete_instance

`rv32_disassemble(instr)`

- Da el texto del comando disassemble

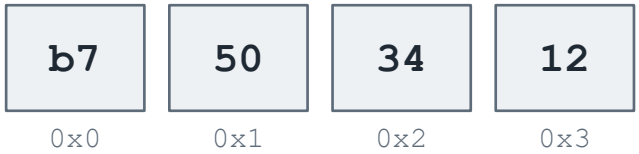
```
# hoy
.word 0x123450b7
# objetivo
lui x1, 0x12345
```

Comandos de Simics

Comando	Para qué
`run`	Ejecuta hasta el ebreak
`run 5 steps / run 7 cycles`	Ejecuta 5 instrucciones o 7 ciclos
`si`	Ejecuta una instrucción
`pregs`	Muestra los registros y el pc
`ptime`	Muestra los steps y los cycles
`disassemble 0x0 3`	Desensambla 3 instrucciones desde 0x0
`x 0x0 16`	Muestra 16 bytes de memoria desde 0x0
`start-command-line-capture f.txt`	Guarda la sesión en un archivo (stop-… para terminar)

Ver la memoria con x

```
simics> x 0x0 16
v:0x00000000  b750 3412 1701 0000 9301 4006 1302 90ff
```



Logging y breakpoints

Mensajes desde C

```
SIM_LOG_INFO(2, cpu->obj, 0,
             "pc=0x%08x", cpu->pc);
```

```
simics> log-level core0 2
```

Mejor que printf: sale junto con la salida de Simics y se activa por nivel.

Detenerse en un paso o un ciclo

```
simics> bp.step.break cosim0 100
simics> bp.cycle.break cosim0 250
simics> run
```

Se detiene después de 100 pasos o 250 ciclos más, lo que llegue primero.

Cómo probar sus procesadores



`all-instructions.txt`

Una primera prueba, no la última.



Sus propios programas

Escriban programas y comparen los resultados con lo que calculan a mano.



Para el core pipelined

Programas con instrucciones que dependen de la anterior, y con bifurcaciones y saltos.

Problemas comunes

Síntoma	Causa probable
Simics se enciclo, congeló, colgó	Falta increment_cycle()
ASSERT en la cola de eventos	increment_step() dos veces en una llamada
Los cambios no se notan	No recompilaron, o no reiniciaron Simics

Demo en vivo

1. Compilar el código
2. Iniciar el target
3. Ver la memoria con x y con disassemble



¿Preguntas?